

Memory Efficient Doubly Linked List
–Destroy List – Program Analysis and Time Complexity

Program:

```
void destroyList()
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        free(curr);
        prev = curr;
        curr = next;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}

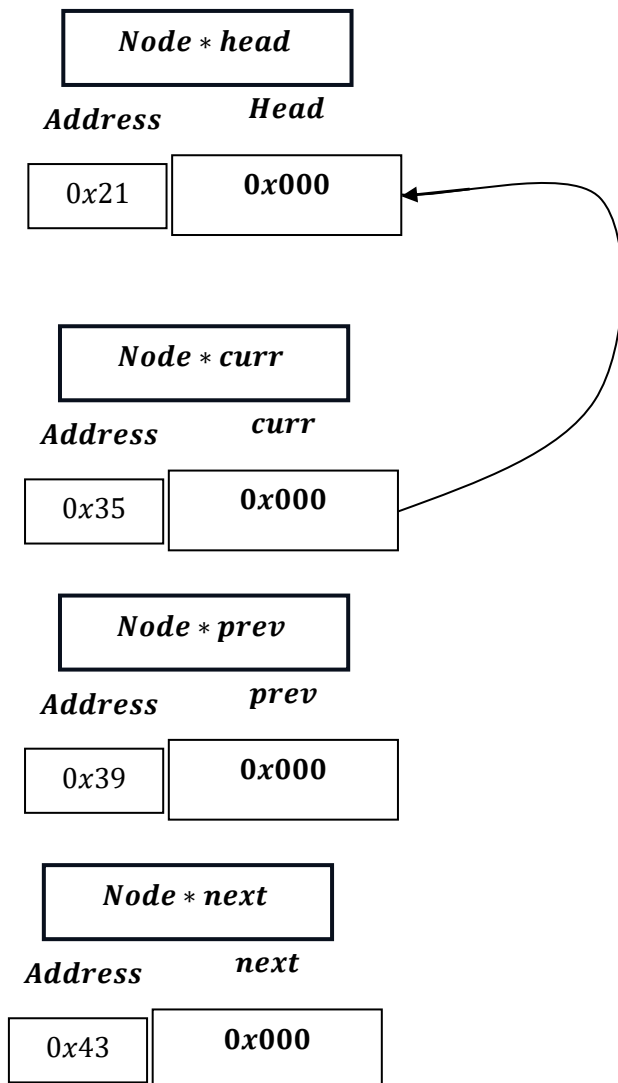
... .

    case 9:
        destroyList();
        break;
```

Program Analysis:

```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```

1. When List Is Empty



```

while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    free(curr);
    prev = curr;
    curr = next;
}
head = nullptr;
cout << "List destroyed." << endl;

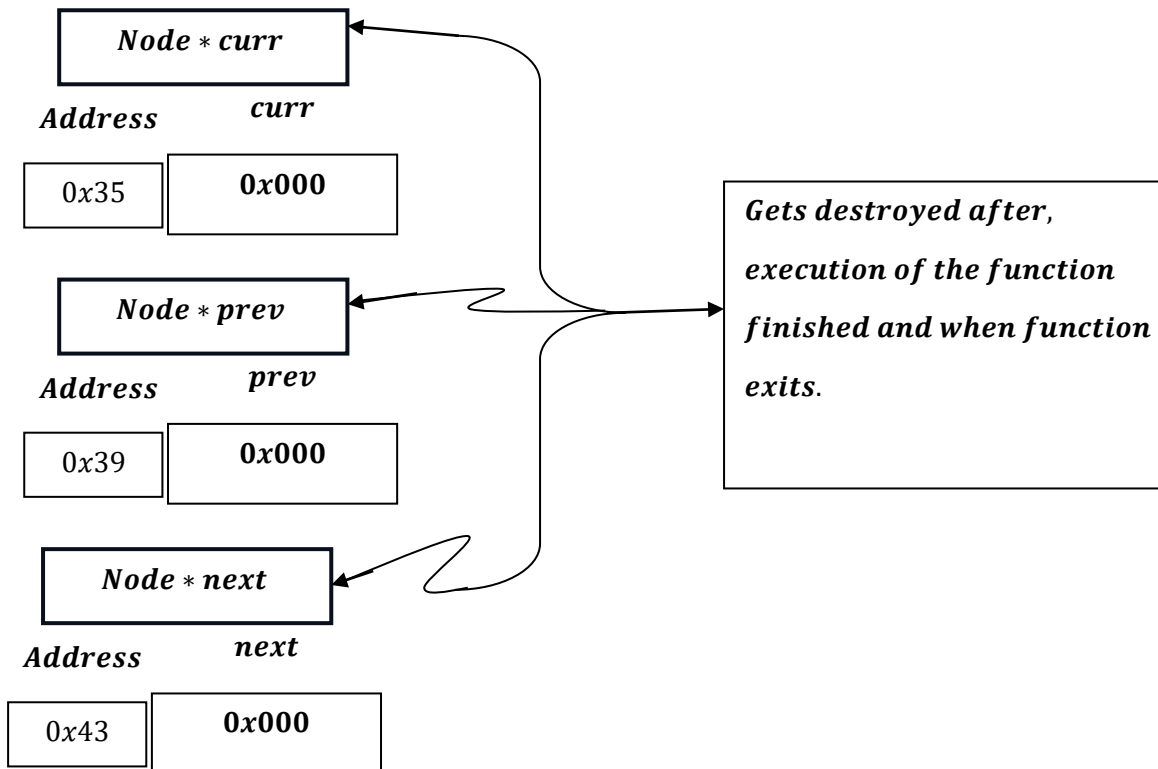
```

while(curr: 0x000 ≠ nullptr: 0x000) → False.

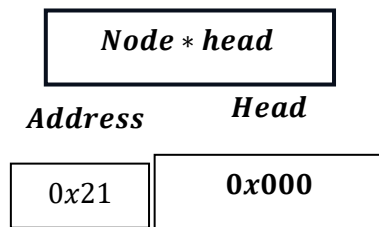
head = nullptr. $\left[\begin{array}{c} \text{just an re - assignment, here no need to show} \\ \text{in a picture.} \end{array} \right]$

Print: ``List destroyed.``

As , functions ends:

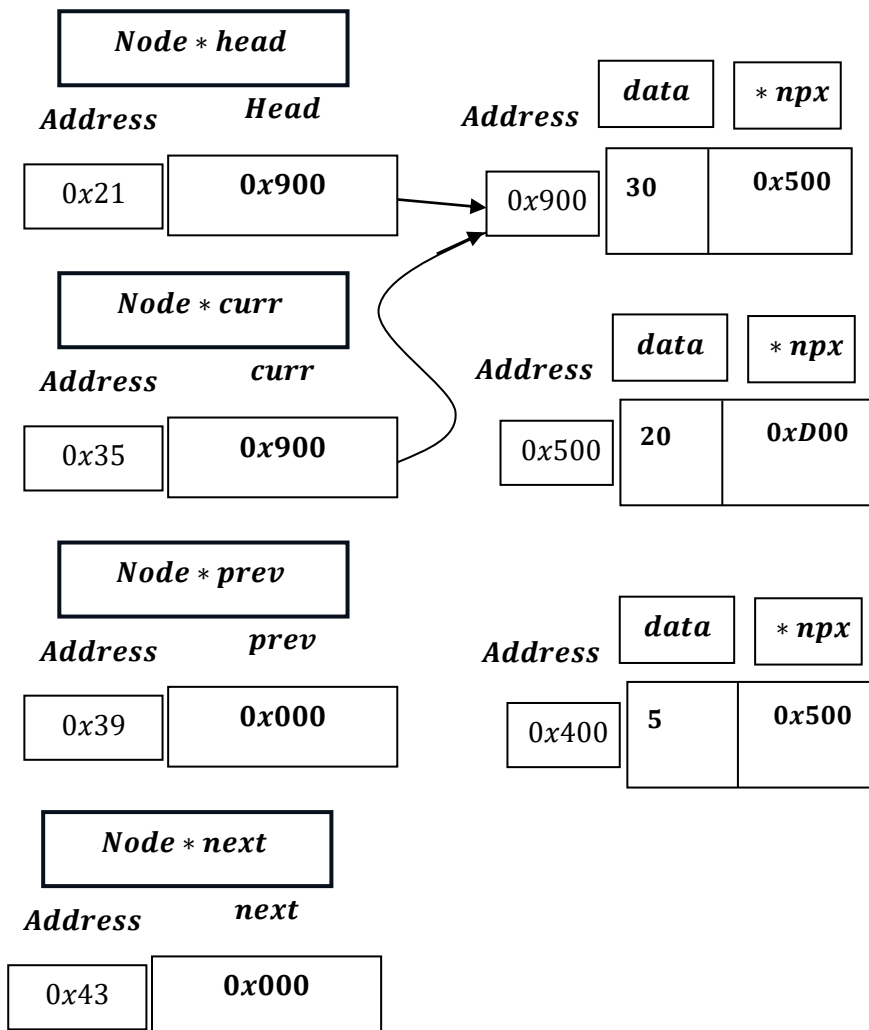


And we are left with:



2. When List Is Not Empty

```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```



```

while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    free(curr);
    prev = curr;
    curr = next;
}

```

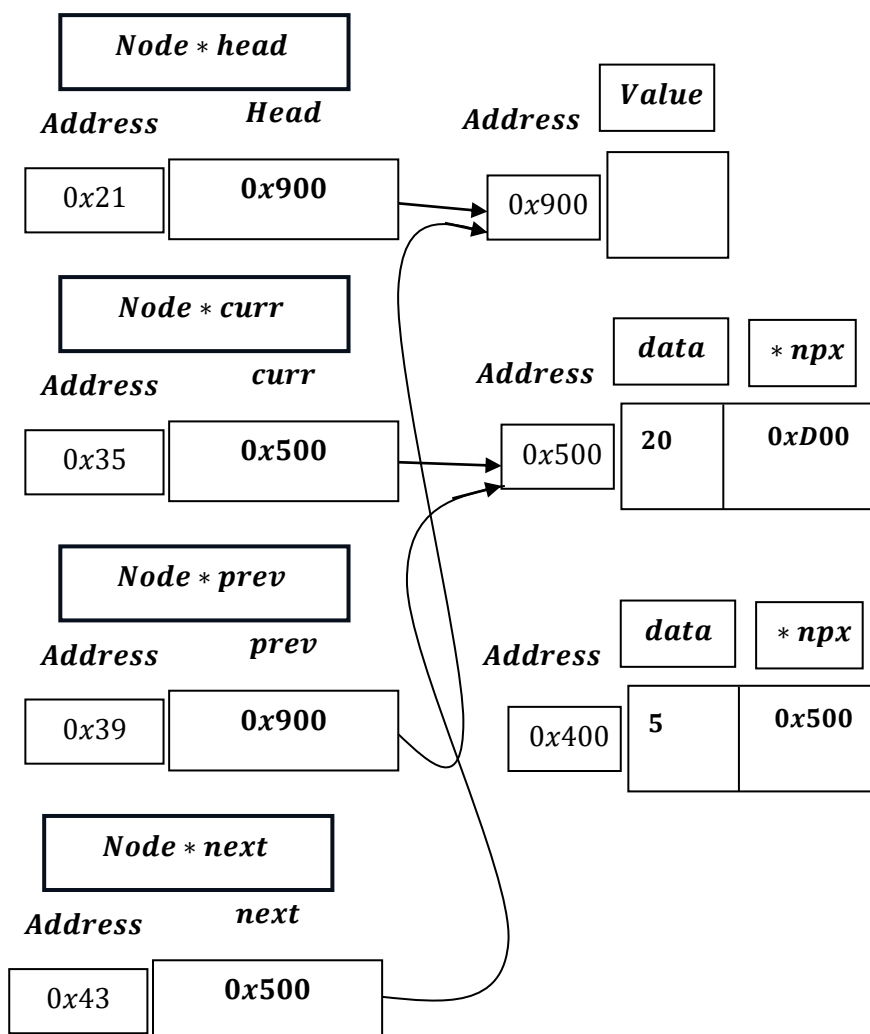
curr: 0x900 $\neq$ nullptr: 0x000 := is True:

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x500

free(curr);

prev = curr = 0x900

curr = next = 0x500



curr: 0x500 $\neq$ nullptr: 0x000 := is True:

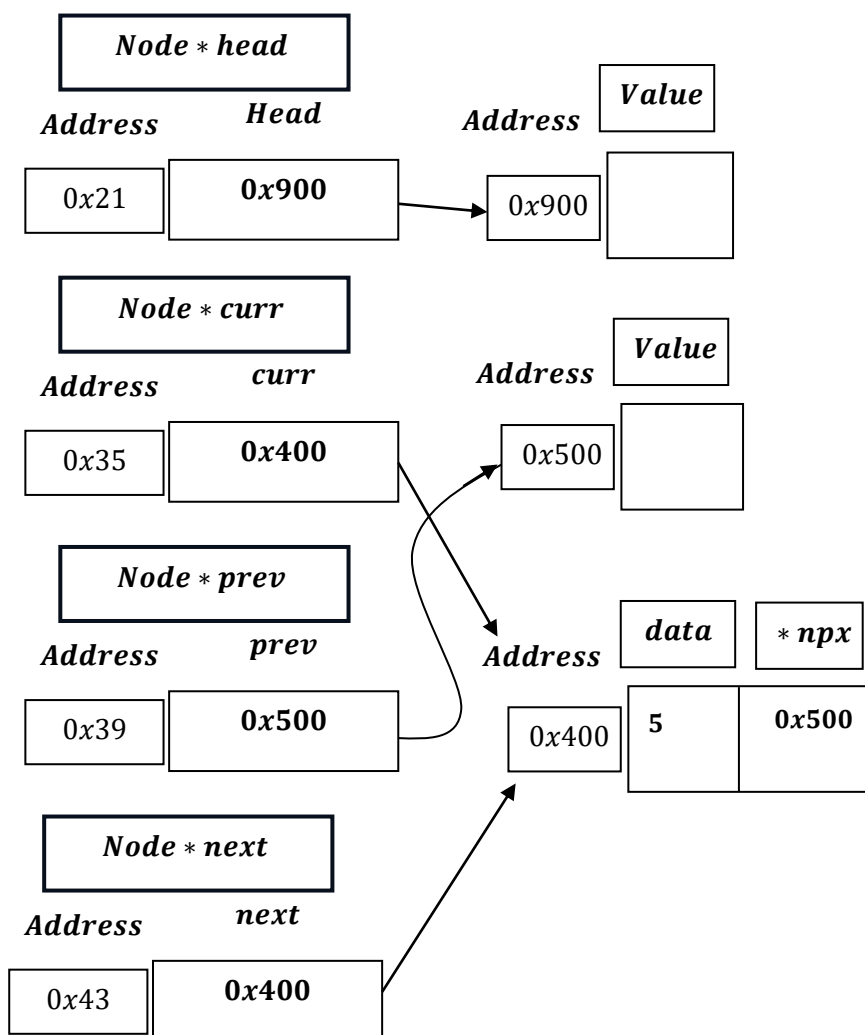
next = XOR(prev: 0x900 , curr $\rightarrow$ npx: 0xD00)

= $1001_2 \oplus 1101_2 = 0100_2 = 4_{10} = 0x400$

free(curr);

prev = curr = 0x500

curr = next = 0x400



curr: 0x400 $\neq$ nullptr: 0x000 := is True:

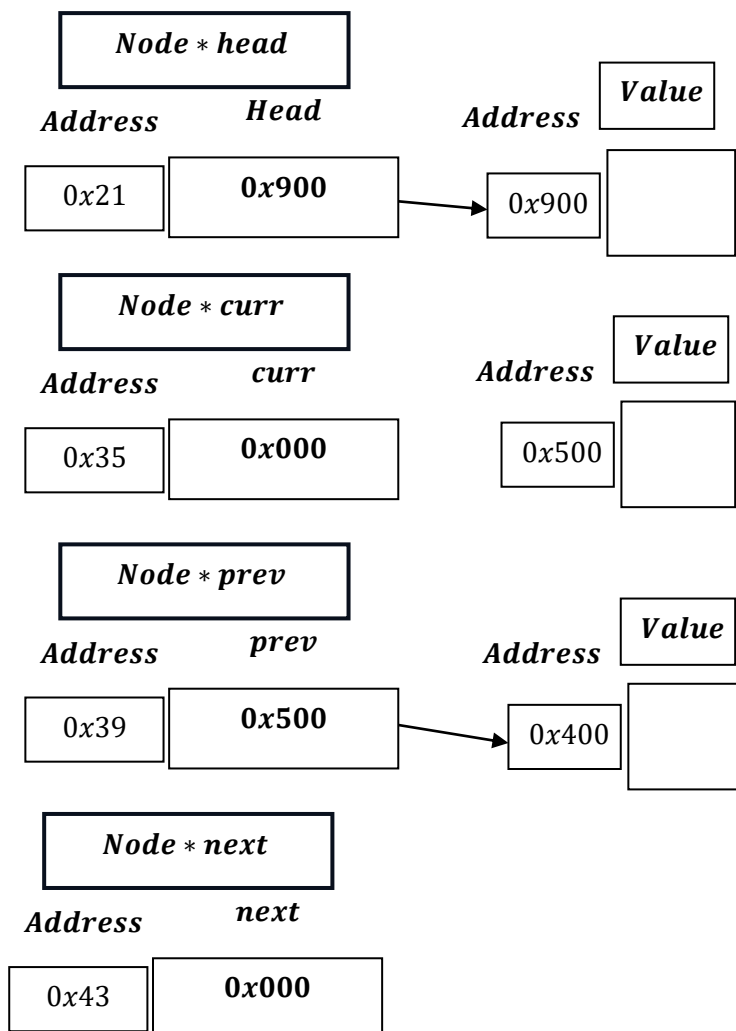
next = XOR(prev: 0x500 , curr $\rightarrow$ npx: 0x500)

= 0101₂ $\oplus$ 0101₂ = 0000₂ = 0₁₀ = 0x000

free(curr);

prev = curr = 0x400

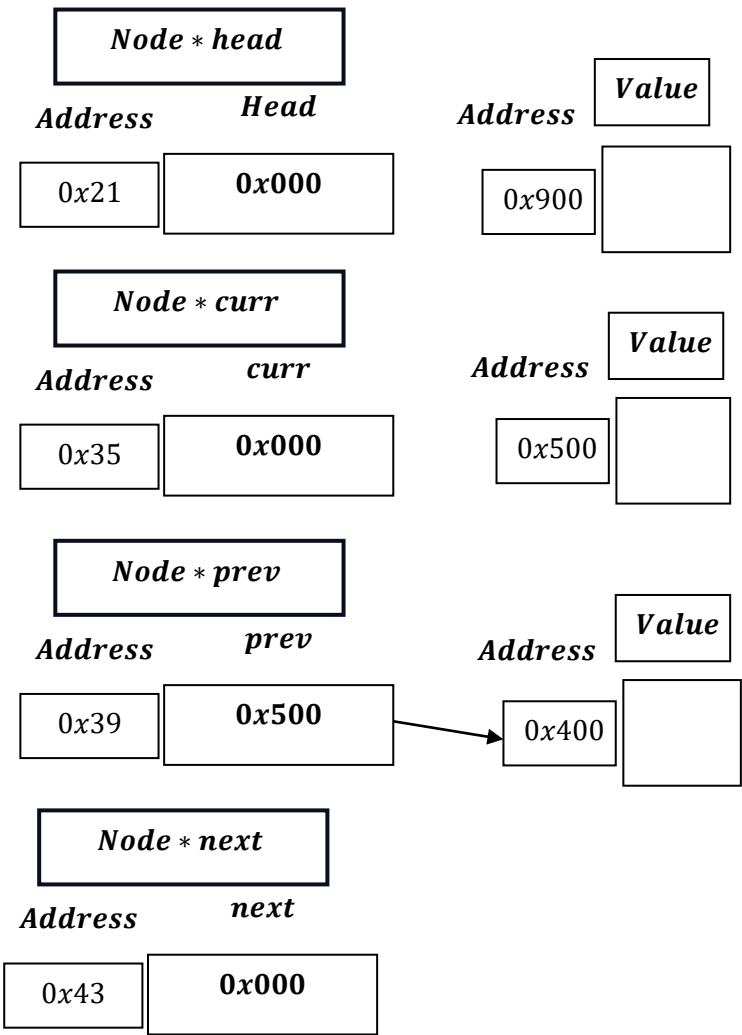
curr = next = 0x000



curr: 0x000 $\neq$ nullptr: 0x000 := is False, hence exit from loop.

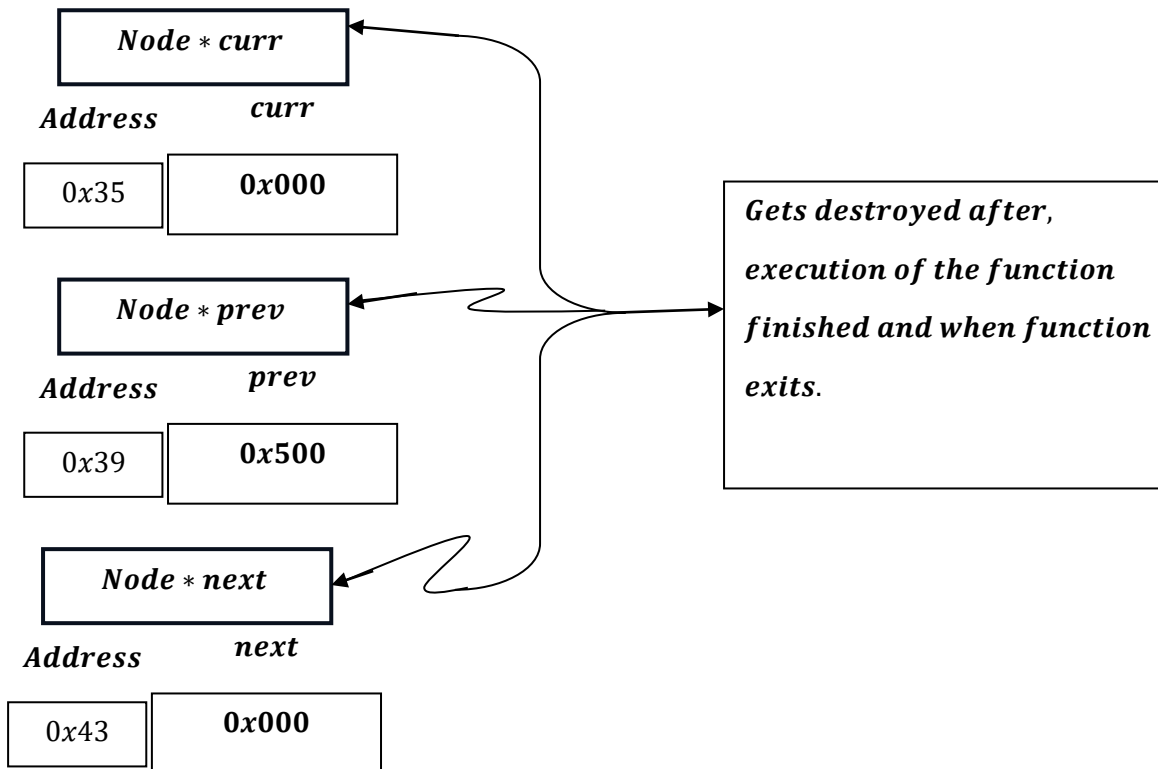

```
head = nullptr;
```

```
head = nullptr = 0x000;
```

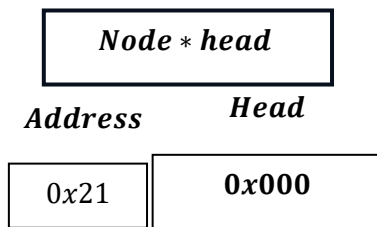


```
Print: Output: ``List Destroyed.``
```

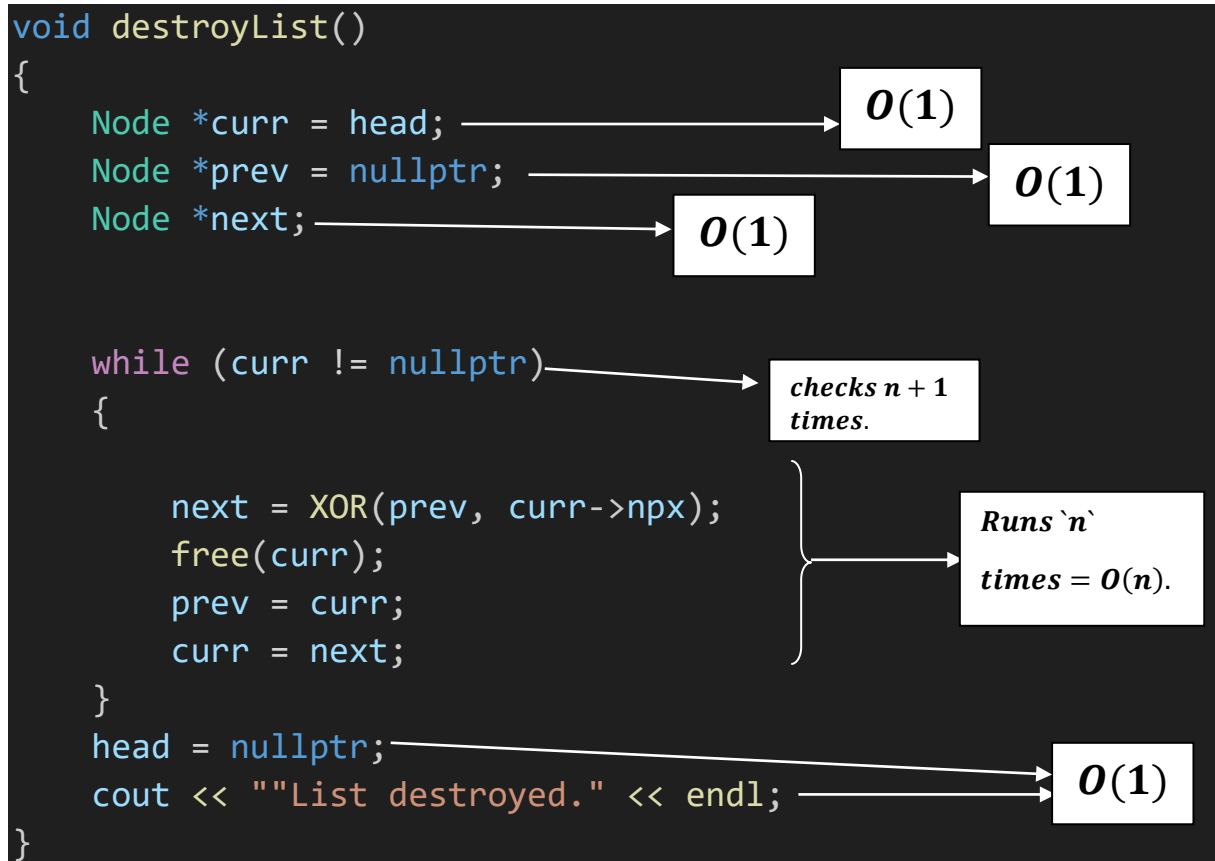
As , functions ends:



And we are left with:



Time Complexity



Best Case

When, List is empty , then we get our absolute best case: $\Omega(1)$.

Worst Case

Full linked list traversal:

$curr \neq nullptr$ $\left[\begin{array}{l} \text{This check runs } n + 1 \text{ times,} \\ \text{including when it becomes false.} \end{array} \right]$:

$next = XOR(prev, curr \rightarrow npx); \rightarrow$ runs : $1 + 1 + 1 + \dots n \text{ times} = n$.

$free(curr); \rightarrow$ runs : $1 + 1 + 1 + \dots n \text{ times} = n$.

$prev = curr; \rightarrow$ runs: $1 + 1 + 1 + \dots n \text{ times} = n$.

$curr = next; \rightarrow$ runs: $1 + 1 + 1 + \dots n \text{ times} = n$.

As , we focus , to the run of inner – statements of loop, hence :

$\therefore \text{Runs} = n + n + n + n + n = 5n = 5 \times O(n) = O(n)$.

Average Case

We can say , if List is Empty ,

, we get our best case scenario, and when list is not empty and there present multiple nodes , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.

In the case of DestroyList, where list is traversed till it reach the end of list, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case).

Since the traversal position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence DestroyList doesn't have a true average case.

No meaningful probabilistic average exists here.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Destroy List</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$
